IT Service Desk & Support Management System

1. Project Overview

The **IT Service Desk & Support Management System** is a backend application for managing IT support requests within an organization.

Employees can report technical problems or request IT services. Support agents can receive, manage, assign, update, and resolve these requests.

The system should maintain complete information about each support ticket, including its status, assigned support agent, comments, attachments, and activity history.

The application must be developed as a **RESTful API** using:

- **Node.js**
- **Express.js**
- **MySQL**

The project should be developed with proper validation, authentication, authorization, database design, file handling, email notifications, and API testing.

2. Main Objective

The objective of this project is to develop a professional backend system that can be used by an organization to manage its internal IT support operations.

The system should allow:

- Employees to create and track support tickets.
- Support agents to manage and resolve tickets.
- Managers/Admins to manage users, teams, categories, and tickets.
- Users to communicate through ticket comments.
- Users to upload relevant files and documents.
- The system to send email notifications for important ticket activities.
- The system to maintain a history of important actions performed on tickets.

3. Technology Requirements

The project must use:

Backend

- Node.js
- Express.js

Database

- MySQL

Authentication

- Google Oauth
- JWT

API Testing

- Postman or any other

API Documentation

- Swagger/OpenAPI is recommended (**Optional**)

Other libraries and packages may be used when necessary.

4. User Roles

The system should have at least the following user roles:

4.1 Employee

Employees can:

- Log in using Google.
- View their profile.
- Create support tickets.
- View their own tickets.
- View ticket details.
- Add comments to their tickets.
- Upload attachments.
- Respond to support requests.
- View ticket status and history.
- Reopen eligible resolved/closed tickets.

4.2 Support Agent

Support agents can:

- Log in using Google.
- View tickets assigned to them.
- View tickets belonging to their support team.
- View ticket details.
- Update ticket status.
- Change ticket priority where permitted.
- Add comments.
- Add internal comments.
- Upload attachments.
- Assign or reassign tickets where permitted.
- Resolve tickets.
- View ticket history.

4.3 Admin / Support Manager

Admins/Support Managers can:

- Manage users.
- Manage support teams.
- Add/remove team members.
- Manage ticket categories.
- Manage ticket priorities.
- View all tickets.
- Assign and reassign tickets.
- Update ticket information.
- Manage support agents.
- View audit logs.
- View ticket history.

- Manage system configuration where applicable.

5. Authentication

Users must authenticate using **Google Oauth** and with **JWT**.

The system should:

- Allow users to sign in with Google.
- Verify the authenticated Google account.
- Create a user account when a new user signs in for the first time.
- Allow existing users to sign in.
- Maintain the user's application session/authentication.
- Provide an endpoint to retrieve the currently authenticated user.
- Provide logout functionality.

The system should not rely only on the frontend for authentication or authorization.

6. User Management

The system should maintain user information such as:

- Name
- Email
- Profile image
- Role
- Account status
- Support team, if applicable
- Created date
- Updated date

Admins should be able to manage user roles and account status where appropriate.

7. Support Ticket

A ticket represents an IT issue or service request submitted by an employee.

A ticket should contain information such as:

- Ticket number
- Title
- Description
- Requester
- Assigned support agent
- Support team
- Category
- Priority
- Status
- Attachments
- Comments
- Created date
- Updated date
- Resolved date
- Closed date

Each ticket should have a unique ticket number.

Example:

IIC-2026-001284

8. Ticket Creation

Employees should be able to create a ticket.

A ticket should require information such as:

- Title
- Description
- Category
- Priority

The requester should automatically be associated with the ticket based on the authenticated user.

The system should validate all submitted information before creating the ticket.

After creation:

- A unique ticket number should be generated.
- The ticket should receive an initial status.
- The ticket should be stored in the database.
- Appropriate notification emails should be sent.

9. Ticket Status

The system should support a ticket workflow.

Suggested statuses:

OPEN

ASSIGNED

IN_PROGRESS

WAITING_FOR_USER

RESOLVED

CLOSED

REOPENED

The backend should enforce valid status changes.

Users should not be able to change a ticket to any status without checking whether the transition is allowed.

For example, an employee should not be able to directly change a ticket to RESOLVED unless the business rules allow it.

10. Ticket Assignment

Support tickets can be assigned to support agents.

The system should support:

- Assigning a ticket to an agent.
- Reassigning a ticket.
- Associating tickets with support teams.

- Viewing tickets assigned to an agent.
- Viewing tickets belonging to a support team.

Only users with appropriate permissions should be able to assign or reassign tickets.

11. Ticket Priority

The system should support different priority levels.

For example:

LOW

MEDIUM

HIGH

URGENT

The priority should help support staff determine which tickets require faster attention.

Admins/authorized support staff should be able to change ticket priority.

12. Ticket Categories

Tickets should belong to a category.

Example categories:

Hardware

Software

Network

Account Access

Email

Security

Other

Admins should be able to:

- Create categories.
- Update categories.
- Enable/disable categories.

- Delete categories where appropriate.

The final categories may be decided during development.

13. Ticket Comments

Users should be able to communicate through comments attached to a ticket.

For example:

Employee:

"My laptop shuts down whenever I connect an external monitor."

Support Agent:

"Please provide the laptop model and attach a screenshot."

Employee:

"The laptop model is Dell Latitude 5440."

The system should record:

- Comment
- Author
- Ticket
- Created date
- Updated date, if editing is allowed

14. Internal Comments

Support agents should be able to create **internal comments**.

Internal comments are visible only to authorized support staff and should not be visible to the employee who created the ticket.

Example:

Internal Comment:

"Possible hardware failure. Need to check the device physically."

The backend must enforce proper access control for internal comments.

15. File Attachments

Users should be able to attach files to tickets.

Possible attachments include:

- Images
- Screenshots
- PDF files
- Documents
- Log files

The system should handle:

- File type validation
- File size validation
- Unique file names
- Upload information
- File association with tickets
- Secure file access
- File deletion where permitted

The system should store information about each uploaded file, including:

- Original file name
- Stored file name
- File type
- File size
- Uploaded by
- Ticket
- Upload date

16. Email Notifications

The system should send email notifications for important events.

Examples:

Ticket Created

The employee receives confirmation that their ticket has been created.

Ticket Assigned

The assigned support agent receives a notification.

Ticket Updated

Relevant users may receive a notification when important ticket information changes.

New Comment

The appropriate user/support agent receives a notification when a new comment is added.

Ticket Resolved

The employee receives a notification when their ticket is resolved.

Ticket Reopened

Relevant support staff receive a notification when a ticket is reopened.

The email system should be implemented in a reusable way rather than duplicating email-sending logic throughout the application.

17. Ticket History

The system should maintain a history of important changes made to a ticket.

For example:

Ticket Created

↓

Priority changed: MEDIUM → HIGH

↓

Assigned to Alex

↓

Status changed: OPEN → IN_PROGRESS

↓

Comment added

↓

Status changed: IN_PROGRESS → RESOLVED

The history should record information such as:

- Ticket
- User who performed the action
- Action
- Previous value, where applicable
- New value, where applicable
- Date/time

Users with appropriate permissions should be able to view ticket history.

18. Audit Logs

The system should maintain audit information for important administrative actions.

Examples:

- User role changed.
- User account disabled.
- Ticket reassigned.
- Category created.
- Category updated.
- Support team modified.

Audit records should include information such as:

- User who performed the action
- Action performed
- Related resource
- Date/time
- Relevant changes

Audit logs should not be editable by normal users.

19. Ticket Search

The API should allow tickets to be searched.

For example:

Search by:

- Ticket number

- Title

- Description

Example:

GET /api/tickets?search=laptop

The search functionality should be implemented on the backend.

20. Ticket Filtering

The API should allow tickets to be filtered.

Examples:

Status

Priority

Category

Assigned Agent

Support Team

Requester

Date

Example:

GET /api/tickets?status=OPEN&priority=HIGH

Multiple filters should be supported where appropriate.

21. Pagination

Ticket lists should support pagination.

Example:

GET /api/tickets?page=1&limit=20

The API response should provide useful pagination information.

Example:

```
{
  "data": [],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 100,
    "totalPages": 5
  }
}
```

22. Sorting

The API should support sorting where appropriate.

For example:

GET /api/tickets?sortBy=createdAt&sortOrder=desc

The API should not allow arbitrary database fields to be used unsafely for sorting.

23. Request Validation

All API requests must be properly validated.

Validation should be applied to:

- Request body
- Query parameters
- URL parameters
- File uploads

Examples:

- Required fields
- String length
- Valid email format
- Valid IDs
- Valid status values
- Valid priority values
- Valid category
- File type
- File size

Invalid requests should return appropriate HTTP status codes and useful error messages.

24. Error Handling

The backend should have consistent error handling.

The API should properly handle errors such as:

- Invalid request
- Unauthorized request
- Forbidden request
- Resource not found
- Duplicate data
- Database errors
- Invalid file
- Authentication errors
- Unexpected server errors

The API should return clear and consistent error responses.

25. Authorization

Authentication alone is not sufficient.

The backend must verify whether the authenticated user has permission to perform an action.

For example:

An employee should not be able to:

- View another employee's private ticket.
- View another user's private attachment.
- Modify administrative settings.
- Change their own role.
- Access internal support comments.

A support agent should not automatically have every administrative permission.

Authorization must be enforced on the backend.

26. REST API

The application must expose its functionality through RESTful APIs.

The API should be logically organized into resources such as:

Authentication

Users

Tickets

Comments

Attachments

Categories

Support Teams

Notifications

Audit Logs

Example endpoints may include:

Authentication

GET /api/auth/google

GET /api/auth/google/callback

GET /api/auth/me

POST /api/auth/logout

Tickets

POST /api/tickets

GET /api/tickets

GET /api/tickets/:id

PATCH /api/tickets/:id

Comments

POST /api/tickets/:id/comments

GET /api/tickets/:id/comments

Attachments

POST /api/tickets/:id/attachments

GET /api/tickets/:id/attachments

GET /api/attachments/:id

DELETE /api/attachments/:id

Categories

GET /api/categories

POST /api/categories

PATCH /api/categories/:id

DELETE /api/categories/:id

Support Teams

GET /api/teams

POST /api/teams

PATCH /api/teams/:id

These are examples. The final API structure should be designed by the developer based on the requirements.

27. Database Requirements

The application must use MySQL.

The database should properly represent relationships between:

- Users
- Roles
- Support teams
- Team members
- Tickets
- Categories
- Priorities
- Comments
- Attachments
- Ticket history
- Notifications
- Audit logs

The database should be properly structured and avoid unnecessary duplication.

28. API Testing

The developer must test the APIs using **Postman** or other api testing tools.

A Postman collection should contain the project's APIs organized into folders.

Example:

IT Service Desk API

│

├── Authentication

│

├── Users

│

├── Tickets

│

├── Comments

│

├── Attachments

│

├── Categories

│

├── Support Teams

│

└── Admin

The collection should demonstrate successful requests as well as important failure cases.

Examples:

- Invalid authentication
- Missing required fields
- Invalid ticket ID
- Unauthorized access
- Forbidden access
- Invalid file type
- Invalid status transition

---

29. API Documentation. (Highly prefer but optional)

The API should be properly documented.

Documentation should explain:

- Endpoint
- HTTP method
- Authentication requirement
- Request parameters
- Request body

- Response
- Possible errors

Swagger/OpenAPI is recommended.

30. Project Structure

The project should have a clean and maintainable structure.

The developer should separate different responsibilities appropriately, such as:

Routes

Controllers

Services

Database/Models

Validation

Middleware

Authentication

File Handling

Email

Utilities

Configuration

The exact project structure is up to the developer.

31. Environment Variables

Sensitive configuration should not be hard-coded into the source code.

Environment variables should be used for information such as:

Database credentials

Google OAuth credentials

Authentication secrets

Email credentials

Application configuration

Sensitive credentials must not be committed to Git.

A .env.example file should be provided.

32. Git & Repository

The project should be maintained in a Git repository.

The repository should have:

- Meaningful commit messages
- Organized source code
- Proper .gitignore
- No passwords or secrets
- README documentation
- Database setup instructions
- Environment variable documentation

33. README Requirements

The project should include a README containing:

Project Overview

What the application does.

Technologies

Technologies and major libraries used.

Installation

How to install dependencies.

Configuration

Required environment variables.

Database Setup

How to create and initialize the database.

Google OAuth Setup

Instructions for configuring Google OAuth.

Running the Application

How to start the development server.

API Documentation

How to access the API documentation.

Postman Collection

How to import and use the Postman collection.

34. Required Deliverables

At the end of the project, the following should be submitted:

1. Source Code

Complete Node.js/Express backend source code.

2. Database

- Database schema
- ER diagram
- Migration/setup scripts
- Seed data where necessary

3. Postman Collection

Complete collection containing the API endpoints.

4. API Documentation

Swagger/OpenAPI or equivalent documentation.

5. README

Complete project setup and usage instructions.

6. Environment Example

A .env.example file showing required configuration variables without exposing actual credentials.

7. Final Demonstration

A demonstration of the completed system, including:

- Google login
- Creating a ticket

- Assigning a ticket
- Updating ticket status
- Adding comments
- Uploading an attachment
- Email notification
- Ticket history
- Role-based access
- API testing through Postman

38. Final Goal

The final application should represent a **professional backend system rather than a basic CRUD project**.

The focus should be on:

Good API design + proper database design + authentication + authorization + business logic + validation + file handling + email notifications + audit/history + API testing.

The developer is expected to make reasonable technical decisions during implementation and should be able to explain those decisions during the final project review. If any question then feel free to contact me.